Rapport Projet RL - Agent Rolling Ball

Auteur: Ajax DESHAYES--HUET

Introduction

Contexte

Ce projet porte sur l'entraînement d'un agent de Reinforcement Learning dans un environnement Unity (ML-Agents), avec un contrôle discret des actions pour déplacer une balle vers une cible (cube).

L'agent est entraîné avec un algorithme Deep Q-Network (DQN), implanté en Python, qui interagit avec Unity via un wrapper Gym en Python.

Objectifs

- Concevoir et compléter un agent DQN fonctionnel.
- Étudier l'impact des hyperparamètres et des récompenses sur l'apprentissage.
- Comparer 4 expériences avec des configurations différentes.
- Interpréter les résultats (performance, stabilité, limites).
- Faire évoluer les objectifs au fil des expériences à partir des résultats observés.

Méthodologie

Logique générale du rapport

Le rapport doit montrer une démarche itérative et non une suite de tests isolés.
Chaque expérience sert à:

1. observer un comportement,
2. l'interpréter,
3. identifier une limite,
4. ajuster les paramètres,
5. vérifier si la nouvelle hypothèse améliore la situation.

Autrement dit, les résultats d'une expérience modifient l'objectif de la suivante.

Paramètres modifiables et hypothèses

- **Gamma (γ)**: poids des rewards futures.
- **N (nombre d'épisodes)**: durée d'entraînement (risque de sous-apprentissage si trop faible).
- **Learning rate (lr)**: vitesse de mise à jour des poids.
- **Epsilon (exploration/exploitation)**: comportement aléatoire vs politique apprise.
- **F (fréquence de mise à jour target network)**.
- **B (batch size)**: taille d'échantillon replay pour chaque update.
- **M (buffer memory)**: capacité de replay memory.
- **NN (taille réseau)**: capacité de représentation, coût de calcul, risque d'instabilité.
- **Récompenses**: signal d'apprentissage (récompense intermédiaire + récompense terminale).

Expériences

> Consigne: lancer **4 expériences** avec des paramètres différents.

Fil conducteur expérimental

Le rapport peut être raconté comme une suite d'itérations:

- Expérience 1: base initiale après complétion des exercices.
- Expérience 2: ajustement pour réduire le lag et accélérer l'expérience.
- Expérience 3: correction d'un sous-apprentissage ou d'une instabilité observée.
- Expérience 4: affinement final, validation de la configuration retenue et réduction des spikes encore trop présents.

Tableau récapitulatif des expériences

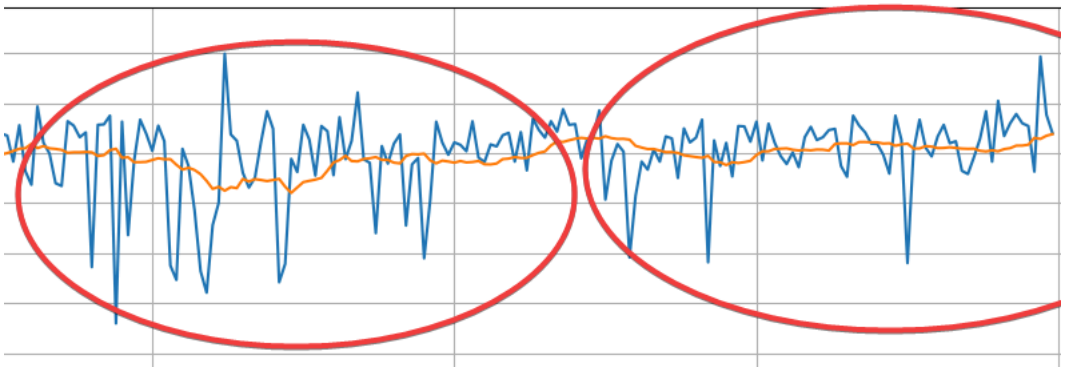
Expérience	Gamma	N épisodes	lr	Epsilon (début/fin/décroissance)	F	B	M	NN	Récompenses	Attente principale
E1 (baseline)	0.99	250	1e-4	0.9 / 0.05 / 2000	500	256	100000	[9,512,512,5]	récompense intermédiaire +/-0.02, terminale +1/-1, limite de temps -0.5	Baseline stable à lancer
E2	0.99	150	7e-5	0.9 / 0.05 / 1500	500	128	100000	[9,512,512,5]	récompense intermédiaire +/-0.01, terminale +1/-1, limite de temps -0.7, limite de pas 400	Réduire le temps de run et les spikes tout en limitant le circling (tourner autour de la récompense)
E3	0.99	220	1e-4	0.9 / 0.02 / 2000	300	128	100000	[9,512,512,5]	récompense intermédiaire +/-0.01, terminale +1/-1, limite de temps -0.5, limite de pas 500	Récupérer stabilité et taux de succès sans changer l'architecture
E4 (finale)	0.99	250	7e-5	0.9 / 0.02 / 2000	300	128	100000	[9,512,512,5]	récompense intermédiaire +/-0.01, terminale +1/-1, limite de temps -0.5, limite de pas 500	Valider le meilleur compromis et lisser les spikes

Détail de chaque expérience (à dupliquer)

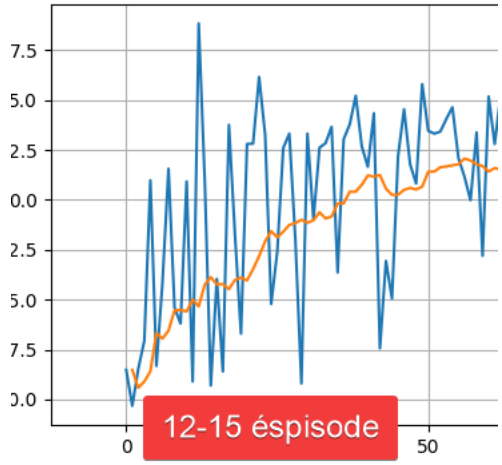
Expérience E1 - Baseline de référence

- **Choix des paramètres**:
  - Configuration actuelle du projet (baseline): `gamma=0.99` , `N=250` , `lr=1e-4` , `epsilon=0.9/0.05/2000` , `F=500` , `batch_size=256` , `memory=100000` , réseau `[9,512,512,5]` .
  - Schéma de récompenses: récompense intermédiaire `+0.02/-0.02` , récompense terminale `+1/-1` , pénalité de limite de temps `-0.5` .
- **Méthodologie / réflexion / approche**:
  - Cette expérience sert de point de référence pour comparer toutes les expériences suivantes.
- **Attentes avant exécution**:
  - Vérifier la stabilité globale de la courbe reward et la tendance de durée moyenne des épisodes.
- **Résultats observés**:

  - Le run est complet: `episodes_completed=250` , `total_steps=47564` , `run_total_seconds=1247.941` .
  - Les moyennes globales sont positives: `mean_episode_reward=1.778` , `mean_episode_duration=190.256` .
  - En fin de run, la courbe montre une stabilisation relative autour d'une tendance positive.



  - En exécution locale (Unity Editor), des ralentissements ont été observés sur la machine pendant certains passages de l'entraînement.
  - La courbe reward monte progressivement en début d'entraînement, puis se stabilise avec une moyenne lissée positive et des pics résiduels.
  - La durée des épisodes baisse globalement, mais reste très variable avec des pics tout au long du run.
  - Des écarts brusques apparaissent entre épisodes consécutifs (ex. épisodes 12-15), ce qui est compatible avec l'aléa du spawn de la cible et l'exploration epsilon-greedy.

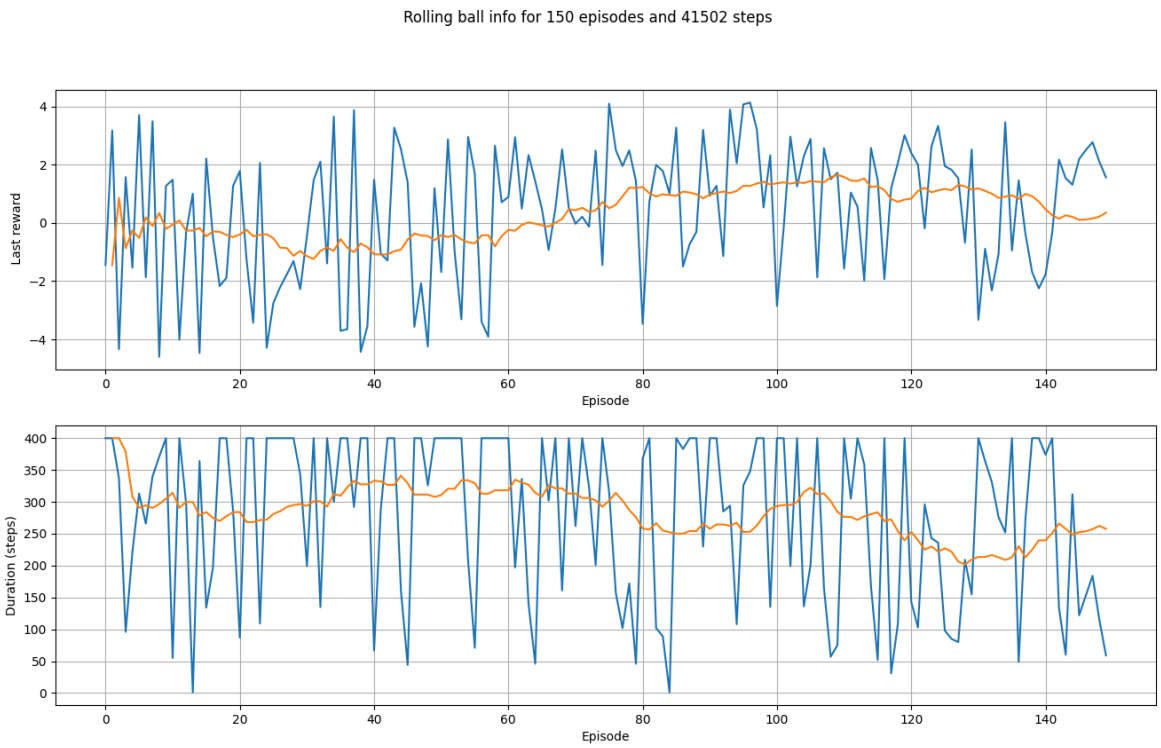


- Attention d'interprétation: le label du graphe indique "Last reward", mais la série tracée correspond en pratique à la reward cumulée par épisode.

- **Interprétation rapide**:
  - E1 valide la base DQN: l'agent apprend à atteindre la cible plus régulièrement en fin de run.
  - Les spikes ne signifient pas un bug en soi; ils sont attendus dans un environnement stochastique avec exploration.
  - Le risque principal est que l'agent fasse du circling (tourne autour de la récompense sans atteindre la cible) pour accumuler des récompenses intermédiaires sans succès terminal suffisamment fiable, ce qui motive une réduction de la récompense intermédiaire et une limite de temps plus pénalisée en E2.

Expérience E2 - Compromis vitesse/stabilité

- **Choix des paramètres**:
  - Hyperparamètres Python: `gamma=0.99` , `N=150` , `lr=7e-5` , `epsilon=0.9/0.05/1500` , `F=500` , `batch_size=128` , `memory=100000` , réseau `[9,512,512,5]` .
  - Schéma de récompenses Unity: récompense intermédiaire `+0.01/-0.01` (au lieu de `+/-0.02`), récompense terminale `+1/-1` conservée, pénalité de limite de temps `-0.7` (au lieu de `-0.5`) et limite d'épisode `400` pas (au lieu de `500`).
  - Motivation principale: pendant E1, la simulation Unity a montré des ralentissements (lag), donc E2 privilégie un compromis vitesse/stabilité avant de pousser la performance brute.
- **Méthodologie / réflexion / approche**:
  - E2 cible trois objectifs opérationnels: réduire le temps de simulation, atténuer les oscillations, et limiter le circling (tourner autour de la récompense sans atteindre la cible).
  - Les changements sont volontairement modérés pour rester comparables à E1 et isoler les effets principaux.
  - Justification des changements liés au lag:
    - `N=150` réduit directement la durée totale du run et accélère le cycle test-analyse.
    - `batch_size=128` diminue le coût de chaque mise à jour DQN (moins de calcul par backward pass), ce qui aide à limiter la charge CPU/GPU pendant l'entraînement dans l'Editor Unity.
    - `max steps=400` évite des épisodes très longs peu informatifs, ce qui réduit aussi le temps de simulation.
- **Attentes avant exécution**:
  - Diminution du temps total de run grâce à `N` plus faible, `batch_size` plus petit et épisodes plus courts.
  - Courbe reward potentiellement un peu moins explosive (moins de points de récompense intermédiaire par pas + `lr` plus faible).
  - Baisse des épisodes longs sans succès terminal, grâce à une limite de temps plus stricte et plus pénalisée.
  - Vérification attendue du lag: baisse de `run_total_seconds` et de `mean_episode_elapsed_seconds` par rapport à E1.
- **Résultats observés**:



- La courbe reward montre une stabilisation faible autour de 0.
- Run complet: `episodes_completed=150` , `total_steps=41502` , `run_total_seconds=842.105` .
- Moyennes globales: `mean_episode_reward=0.384` , `mean_episode_duration=276.68` , `mean_episode_elapsed_seconds=5.574` .
- Les spikes restent importants sur la reward et sur la durée; la stabilisation est moins nette que prévu.
- Beaucoup d'épisodes se terminent au timeout (temps écoulé) (`terminal_reward=-0.7`): 59/150 (39.33%).
- Le succès existe mais reste partiel (`terminal_reward=1.0`): 63/150 (42.00%).
- Observation pratique post-run: une partie du lag perçu venait du contexte d'exécution (focus de la fenêtre terminal), pas uniquement des hyperparamètres.

- **Interprétation rapide**:
  - E2 a bien réduit le temps total de run par rapport à E1, mais au prix d'une baisse de qualité d'apprentissage (reward moyenne plus faible, épisodes plus longs, nombreux timeouts).
  - La combinaison `eps_decay=1500 + limite de pas=400 + limite de temps=-0.7` paraît trop contraignante pour converger proprement.
  - Le lag reste contraignant en pratique, donc E3 conserve un compromis `batch_size=128` et corrige d'abord les autres facteurs (limite de temps, epsilon final, nombre d'épisodes).

Expérience E3 - Stabilisation après E2

- **Choix des paramètres**:
  - Hyperparamètres Python: `gamma=0.99` , `N=220` , `lr=1e-4` , `epsilon=0.9/0.02/2000` , `F=300` , `batch_size=128` , `memory=100000` , réseau `[9,512,512,5]` .
  - Schéma de récompenses Unity: récompense intermédiaire `+0.01/-0.01` conservée, récompense terminale `+1/-1` conservée, pénalité de limite de temps `-0.5` et limite d'épisode `500` pas.
- **Méthodologie / réflexion / approche**:
  - E3 corrige E2 avec une stratégie conservative: ne pas toucher l'architecture du réseau, conserver `eps_decay=2000` et retenir un compromis de calcul avec `B=128` pour éviter les ralentissements sévères observés à `B=256` .
  - Le paramètre `F` est abaissé de `500` à `300` pour synchroniser plus fréquemment le target network, afin de réduire certaines oscillations observées en E2 sans introduire une rupture majeure de configuration.
  - `N` passe à `220` pour donner plus de temps d'apprentissage que E2 (150 épisodes) sans revenir au coût complet de E1 (250 épisodes).
  - `eps_end` passe de `0.05` à `0.02` pour réduire l'aléatoire en fin d'entraînement et stabiliser la politique finale.
  - Le but est d'isoler l'effet des contraintes trop strictes d'E2 (limite de temps courte et pénalité forte) qui ont probablement augmenté les échecs à 400 pas.
- **Attentes avant exécution**:
  - Augmentation du taux de succès et baisse du taux de timeout.
  - Courbe reward plus lisible (moins de saturation autour des épisodes coupés à 400).

- Durée de run raisonnable (N=220) tout en laissant plus de temps d'apprentissage que E2.

- **Résultats observés**:

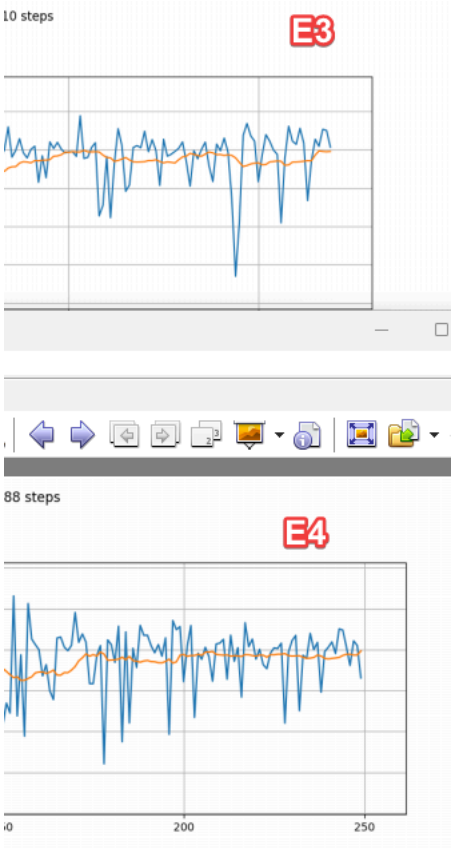
  - Le run est complet: `episodes_completed=220` , `total_steps=54110` , `run_total_seconds=1481.531` .
  - Moyennes globales: `mean_episode_reward=0.5226` , `mean_episode_duration=245.95` , `mean_episode_elapsed_seconds=6.696` .
  - Répartition des terminaisons:
    - succès (`terminal_reward=1.0`): 123/220 (55.9%).
    - timeout (`terminal_reward=-0.5`): 37/220 (16.8%).
    - échec (`terminal_reward=-1.0`): 60/220 (27.3%).
  - La courbe reward progresse globalement mais reste volatile, avec des spikes encore visibles.

- **Interprétation rapide**:
  - E3 corrige une partie des effets négatifs observés en E2.
  - Le taux de succès remonte nettement (55.9% vs 42.0% en E2) et les timeouts baissent fortement (16.8% vs 39.33% en E2).
  - E3 reste toutefois en dessous de E1 sur la performance globale (reward moyenne et régularité de convergence).
  - Le compromis E3 est valide pour stabiliser sans exploser le coût de calcul, mais ne dépasse pas encore la baseline E1.

Expérience E4 - Finale (validation du compromis)

- **Choix des paramètres**:
  - Configuration finale de validation: `gamma=0.99` , `N=250` , `lr=7e-5` , `epsilon=0.9/0.02/2000` , `F=300` , `batch_size=128` , `memory=100000` , réseau `[9,512,512,5]` .
  - Schéma de récompenses Unity: récompense intermédiaire `+0.01/-0.01` , récompense terminale `+1/-1` , pénalité de limite de temps `-0.5` , limite d'épisode `500` pas.
- **Méthodologie / réflexion / approche**:
  - Cette expérience sert de validation finale de la meilleure configuration candidate, avec un apprentissage un peu plus prudent pour lisser les spikes encore trop présents.
- **Attentes avant exécution**:
  - Vérifier si une baisse du learning rate et un temps d'apprentissage plus long réduisent les oscillations sans détruire le taux de succès.
- **Résultats observés**:

- Le run final est complet: `episodes_completed=250` , `total_steps=63588` , `run_total_seconds=1396.504` .
- Moyennes globales: `mean_episode_reward=0.9013` , `mean_episode_duration=254.352` , `mean_episode_elapsed_seconds=5.5687` .
- Répartition des terminaisons:
  - succès (`terminal_reward=1.0`): 152/250 (60.8%).
  - timeout (`terminal_reward=-0.5`): 39/250 (15.6%).
  - échec (`terminal_reward=-1.0`): 59/250 (23.6%).
- La courbe reward reste variable, mais la fin de run est plus lisible que E3 avec moins de spikes marquants.



- **Interprétation rapide**:
  - E4 améliore nettement E3 sur la reward moyenne et le taux de succès, tout en conservant un taux de timeout bas.

- Le compromis visé (stabilité sans effondrement des performances) est globalement atteint.
- E4 ne dépasse pas E1 en performance brute, mais offre un équilibre plus prudent et plus robuste que E2/E3.

Résultats

Cette section sert de synthèse globale. Les détails d'observation et d'interprétation sont documentés dans chaque expérience.

Résultats attendus vs observés (synthèse)

- E2 atteint l'objectif de réduction du temps de run, mais dégrade la qualité d'apprentissage (reward, succès, timeouts).
- E3 récupère une partie de la stabilité perdue en E2, sans retrouver le niveau global de E1.
- E4 améliore encore le compromis (reward et succès en hausse vs E3), avec une fin de run plus lisible.
- Le couple limite de temps stricte + pénalité forte en E2 paraît trop contraignant, et le runtime reste sensible aux conditions machine.

Tableau de comparaison finale

Critère	E1	E2	E3	E4
Reward moyenne (fin de run)	1.778	0.384	0.523	0.901
Durée moyenne épisode	190.256	276.68	245.95	254.352
Stabilité (spikes)	Moyenne	Faible (spikes fréquents)	Moyenne-faible (amélioration vs E2)	Moyenne (moins de spikes en fin de run vs E3)
Taux de succès (si mesuré)	72.4%	42.0%	55.9%	60.8%

Analyse

Les résultats confirment un compromis net entre vitesse de simulation et stabilité d'apprentissage: E2 accélère le run mais dégrade la convergence, alors que E3 puis E4 rétablissent progressivement la stabilité sans revenir aux coûts les plus élevés. La récompense intermédiaire réduite aide à limiter le circling, mais ne suffit pas seule si les contraintes de temps sont trop sévères. Enfin, l'interprétation reste limitée par un nombre restreint de runs, l'aléa du spawn, et la sensibilité au contexte d'exécution (Editor/charge machine).

Conclusion

- E1 reste la meilleure performance brute observée.
- E4 est la configuration recommandée pour le meilleur compromis stabilité/performance sur ce projet.
- Les objectifs initiaux sont atteints: agent fonctionnel, impact des hyperparamètres observé, comparaison des 4 expériences complète.
- Pour consolider ces conclusions: répéter les runs avec seed fixée, puis tester Double DQN si nécessaire.